

HOM Studio OS v2: PRD and Implementation Roadmap

Clinical Studio Operating System with AI Knowledge Studio

Prepared for Arjuna

2026-05-25

Contents

1 Executive Summary	7
2 How to Use This Document	8
2.1 Beginner Glossary	8
3 Current System Audit	8
3.1 What Already Exists Based on Screenshots	8
3.2 Strengths	9
3.3 Red Flags	9
4 Product Requirements	9
5 01 - Product Requirements Document	9
6 HOM Studio OS v2	9
6.1 1. Product Summary	9
6.2 2. Product Vision	9
6.3 3. Main Problems	9
6.3.1 Problem 1 - Operational Data Is Spread Out	9
6.3.2 Problem 2 - Finance Needs One Source of Truth	9
6.3.3 Problem 3 - AI Must Be Useful but Controlled	10
6.3.4 Problem 4 - Clinical Context Requires More Care Than Fitness Booking	10
6.3.5 Problem 5 - Owner Needs No-Code Knowledge Control	10
6.4 4. Target Users	10
6.5 5. Product Goals	10
6.6 6. Non-Goals	10
6.7 7. MVP Feature Scope	11
6.7.1 Phase 1 - Foundation	11
6.7.2 Phase 2 - Clinical Operations	11
6.7.3 Phase 3 - Finance	11
6.7.4 Phase 4 - Communication	11
6.7.5 Phase 5 - Knowledge Studio	11
6.7.6 Phase 6 - Behavior Intelligence	11
6.7.7 Phase 7 - Executive Intelligence	12
6.8 8. Functional Requirements	12
6.8.1 8.1 Appointment Module	12
6.8.2 8.2 Clinical Case Registry	12
6.8.3 8.3 Session Notes	12
6.8.4 8.4 Finance	13
6.8.5 8.5 WhatsApp Inbox	13
6.8.6 8.6 Knowledge Studio	13
6.8.7 8.7 AI Business Agent	14

6.9	9. Non-Functional Requirements	14
6.9.1	Performance Targets	14
6.9.2	Security Requirements	14
6.9.3	Reliability Requirements	14
6.10	10. Success Metrics	15
6.11	11. Major Risks	15
6.12	12. Out of Scope Until Later	15
7	System Blueprint	15
8	02 - System Blueprint	15
8.1	1. High-Level Architecture	15
8.2	2. Runtime Architecture	16
8.3	3. Core Modules	16
8.4	4. Module Responsibilities	16
8.4.1	auth/	16
8.4.2	roles/	17
8.4.3	clients/	17
8.4.4	practitioners/	17
8.4.5	appointments/	17
8.4.6	clinical-cases/	17
8.4.7	session-notes/	17
8.4.8	finance/	17
8.4.9	commissions/	17
8.4.10	reimbursements/	17
8.4.11	whatsapp/	17
8.4.12	knowledge-studio/	17
8.4.13	behavior-intelligence/	17
8.4.14	ai-gateway/	17
8.4.15	approvals/	18
8.4.16	audit/	18
8.5	5. Data Flow - Appointment Creation	18
8.6	6. Data Flow - WhatsApp AI Draft	18
8.7	7. Data Flow - Knowledge Upload	18
8.8	8. Data Flow - Behavior Learning	19
8.9	9. Data Flow - Finance Report	19
8.10	10. Service Boundaries	19
8.10.1	Core transaction boundaries	19
8.10.2	AI boundaries	20
8.10.3	Worker boundaries	20
8.11	11. Integration Boundaries	20
8.12	12. Scalability Model	20
9	Architecture Decisions	20
10	03 - Architecture Decisions	20
10.1	ADR 001 - Use Modular Monolith, Not Microservices	20
10.1.1	Decision	20
10.1.2	Reason	20
10.1.3	Consequence	20
10.2	ADR 002 - Use Selective Hexagonal Architecture	21
10.2.1	Decision	21
10.2.2	Reason	21
10.2.3	Consequence	21
10.3	ADR 003 - Remove n8n from Critical Path	21
10.3.1	Decision	21
10.3.2	Reason	21
10.3.3	Consequence	21

10.4	ADR 004 - Use Event Outbox and Worker	21
10.4.1	Decision	21
10.4.2	Reason	21
10.4.3	Consequence	22
10.5	ADR 005 - Use Multi-LLM Through a Gateway Only	22
10.5.1	Decision	22
10.5.2	Reason	22
10.5.3	Consequence	22
10.6	ADR 006 - AI Must Be Human-in-the-Loop for Sensitive Actions	22
10.6.1	Decision	22
10.6.2	Reason	22
10.6.3	Consequence	22
10.7	ADR 007 - Use Knowledge Studio Instead of Hardcoded Prompts	22
10.7.1	Decision	22
10.7.2	Reason	23
10.7.3	Consequence	23
10.8	ADR 008 - Deployment Stack	23
10.8.1	Decision	23
10.8.2	Reason	23
10.8.3	Consequence	23
10.9	ADR 009 - Frontend Quality Stack	23
10.9.1	Decision	23
10.9.2	Reason	23
10.9.3	Consequence	23
10.10	ADR 010 - Source of Truth	24
10.10.1	Decision	24
10.10.2	Reason	24
10.10.3	Consequence	24
11	Database Blueprint	24
11.1	Identity and Permissions	24
11.2	Operations	24
11.3	Clinical Workflow	24
11.4	Finance	24
11.5	Communication and AI	25
11.6	Worker and Governance	25
12	AI Knowledge Studio	25
1306	- AI Knowledge Studio and Behavior Intelligence	25
13.11	Goal	25
13.22	Why This Matters	25
13.33	Main Pages	25
13.3.13.1	Knowledge Sources	25
13.3.23.2	Upload Panel	26
13.3.33.3	Extraction Review	26
13.3.43.4	Spreadsheet Mapping	26
13.3.53.5	Business Rules Editor	27
13.3.63.6	Chatbot Behavior Profile	27
13.3.73.7	Test Lab	27
13.3.83.8	Publish and Rollback	28
13.44	Knowledge Scopes	28
13.55	Document Ingestion Pipeline	28
13.66	Parser Recommendation	28
13.77	RAG Retrieval Rules	29
13.88	AI Safety Rules	29
13.99	Behavior Intelligence	29

13.9.19.1 What to Extract from Chat	29
13.9.29.2 Example	29
13.9.39.3 Behavior Dashboard	30
13.100. Human Approval Loop	30
13.111. Multi-LLM Gateway	30
13.122. Observability	30
13.133. MVP Build Order	31
14 Worker and Automation Plan	31
1507 - Worker, Queue, and Automation Plan	31
15.11. Goal	31
15.22. Tasks That Must Run in Worker	31
15.33. Recommended MVP Queue	31
15.44. Event Types	32
15.55. Worker Rules	32
15.66. Retry Policy	32
15.77. Dead-Letter Queue	32
15.88. Scheduled Jobs	33
15.99. Workflow Without n8n	33
15.100. Optional n8n Migration Bridge	33
15.111. Worker Dashboard	33
15.122. Performance Benchmarks	33
15.133. Scaling Path	34
16 Frontend UI/UX Plan	34
1708 - Frontend UI/UX Guide	34
17.11. Goal	34
17.22. Recommended Stack	34
17.33. Motion.dev Usage	34
17.44. UIPro Usage	35
17.55. 21st.dev Usage	35
17.66. Design System	35
17.6.1 Colors	35
17.6.2 Typography	36
17.6.3 Spacing	36
17.6.4 Radius and Shadows	36
17.77. Main Pages	36
17.7.17.1 Dashboard Layout	36
17.7.27.2 Strategic Overview	36
17.7.37.3 Appointments	37
17.7.47.4 Live Chat	37
17.7.57.5 Knowledge Studio	37
17.7.67.6 Finance	37
17.7.77.7 Clinical Cases	37
17.7.87.8 Approvals	38
17.88. Component Library	38
17.99. Accessibility Checklist	38
17.100. Frontend Quality Gate	38
18 Deployment and Capacity Plan	39
1909 - Deployment Phases and Capacity Plan	39
19.11. Final Recommendation	39
19.22. Phase 0 - Development Prototype	39
19.2.1 Use When	39
19.2.2 Stack	39

19.2.3	Benchmarks	39
19.2.4	Cost Range	39
19.33.	Phase 1 - MVP Production Internal	39
19.3.1	Use When	39
19.3.2	Stack	40
19.3.3	Benchmarks	40
19.3.4	Cost Range	40
19.44.	Phase 2 - Stable Production	40
19.4.1	Upgrade Triggers	40
19.4.2	Stack	40
19.4.3	Benchmarks	40
19.4.4	Cost Range	40
19.55.	Phase 3 - Heavy AI and Document Processing	41
19.5.1	Upgrade Triggers	41
19.5.2	Stack	41
19.5.3	Benchmarks	41
19.5.4	Cost Range	41
19.66.	Phase 4 - Multi-Studio / SaaS Serious	41
19.6.1	Upgrade Triggers	41
19.6.2	Stack Options	41
19.6.3	Benchmarks	41
19.6.4	Cost Range	41
19.77.	Things People Forget	42
19.7.17.1	AI Cost Can Exceed Hosting	42
19.7.27.2	Region Alignment Matters	42
19.7.37.3	File Storage and Egress	42
19.7.47.4	Observability From Day One	42
19.7.57.5	Backups and Restore	42
19.88.	Final Hosting Decision	42

20 Security and Governance 43

2110 - Security and Governance 43

21.11.	Why Security Matters Here	43
21.22.	Role-Based Access Control	43
21.33.	Sensitive Actions Requiring Audit Log	44
21.44.	AI Governance	44
21.55.	PII and Data Masking	44
21.66.	Knowledge Scope Safety	44
21.77.	Supabase RLS	45
21.88.	Service Role Safety	45
21.99.	Secrets Management	45
21.100.	Logs and Error Tracking	45
21.111.	Approval Workflow	46
21.122.	Minimum Production Checklist	46

22 Testing and Quality Gates 46

2311 - Testing and Quality Gates 46

23.11.	Goal	46
23.22.	Test Types	46
23.2.1	Unit Tests	46
23.2.2	Integration Tests	46
23.2.3	End-to-End Tests	47
23.2.4	Load Tests	47
23.2.5	Visual Tests	47
23.33.	Business Rule Tests	47
23.3.1	Appointment Overlap	47

23.3.2Finance Summary	47
23.3.3Clinical Notes	48
23.3.4AI Policy Guard	48
23.44. Frontend Quality Gates	48
23.55. CI Pipeline	48
23.66. Observability Gates	48
23.77. Definition of Done	48
23.88. Manual QA Checklist	49
23.99. Performance Test Targets	49
23.100. AI Evaluation	49

24Step-by-Step Execution Plan 50

2512 - Step-by-Step Execution Plan 50

25.1How to Use This Plan	50
25.2Sprint 0 - Project Setup and Documentation	50
25.2.1Goal	50
25.2.2Tasks	50
25.2.3Acceptance Criteria	50
25.3Sprint 1 - Auth and RBAC	50
25.3.1Goal	50
25.3.2Tasks	50
25.3.3Acceptance Criteria	51
25.4Sprint 2 - Base UI Shell	51
25.4.1Goal	51
25.4.2Tasks	51
25.4.3Acceptance Criteria	51
25.5Sprint 3 - Clients, Practitioners, Services	51
25.5.1Goal	51
25.5.2Tasks	51
25.5.3Acceptance Criteria	51
25.6Sprint 4 - Appointments Core	51
25.6.1Goal	51
25.6.2Tasks	52
25.6.3Acceptance Criteria	52
25.7Sprint 5 - Clinical Cases and Session Notes	52
25.7.1Goal	52
25.7.2Tasks	52
25.7.3Acceptance Criteria	52
25.8Sprint 6 - Finance Foundation	52
25.8.1Goal	52
25.8.2Tasks	52
25.8.3Acceptance Criteria	53
25.9Sprint 7 - Worker and Event Outbox	53
25.9.1Goal	53
25.9.2Tasks	53
25.9.3Acceptance Criteria	53
25.10Sprint 8 - WhatsApp Inbox	53
25.10.1Goal	53
25.10.2Tasks	53
25.10.3Acceptance Criteria	53
25.11Sprint 9 - AI Gateway	53
25.11.1Goal	53
25.11.2Tasks	54
25.11.3Acceptance Criteria	54
25.12Sprint 10 - Knowledge Studio MVP	54
25.12.1Goal	54

25.12.Tasks	54
25.12.Acceptance Criteria	54
25.13 Sprint 11 - Behavior Intelligence	54
25.13.Goal	54
25.13.Tasks	54
25.13.Acceptance Criteria	55
25.14 Sprint 12 - Executive Dashboard	55
25.14.Goal	55
25.14.Tasks	55
25.14.Acceptance Criteria	55
25.15 Sprint 13 - Production Readiness	55
25.15.Goal	55
25.15.Tasks	55
25.15.Acceptance Criteria	55
26 Codex Execution Prompts	56
27 Risk Register	56
28 First 30 Days Plan	56
28.1 Week 1	56
28.2 Week 2	56
28.3 Week 3	56
28.4 Week 4	57
29 Beginner Checklist Before Coding	57
30 Source and Tool Reference	57
30.1 References Used for Technical Decisions	57

1 Executive Summary

HOM Studio OS v2 is a rebuild plan for a clinical pilates/physio studio operating system. The existing super admin screenshots show a system that is already more than a booking platform: it includes executive dashboard, financials, AI Business Agent, WhatsApp live chat, appointments, clients, chronic case registry, team attendance, user management, approvals, and payroll-related workflow.

The correct rebuild strategy is not a generic Fit Hub clone. It should be a clinical studio operating system with controlled AI assistance, owner-editable knowledge, behavior intelligence, audited finance, and human-in-the-loop governance.

The recommended architecture is:

- AI-native Modular Monolith
- + Selective Hexagonal Architecture
- + Code-first Worker/Queue
- + Multi-LLM Gateway
- + RAG Knowledge Layer
- + Human-in-the-loop Governance

The recommended initial infrastructure is:

- Vercel: frontend
- Render: backend API
- Render: worker
- Render Key Value or Postgres-backed queue: queue/cache
- Supabase: database, auth, storage, pgvector

2 How to Use This Document

This PDF is the strategic and operational guide. The companion Markdown files are Codex-ready execution documents. Use the Markdown files as the working source in your repo.

Start with:

1. 00_MASTER_CONTEXT_FOR_CODEX.md
2. 01_PRD.md
3. 02_SYSTEM_BLUEPRINT.md
4. 12_STEP_BY_STEP_EXECUTION_PLAN.md
5. 13_CODEX_TASK_PROMPTS.md

Do not paste all prompts into Codex at once. Execute one prompt, review diff, test, then commit.

2.1 Beginner Glossary

- PRD: Product Requirements Document. A document that explains what the product must do, who uses it, and how success is measured.
- Blueprint: A technical map of the system: modules, data, integrations, and flows.
- Frontend: The screen/dashboard that users see.
- Backend: The server logic that validates data, runs rules, and connects to database and external services.
- Database: The source of truth where data is stored.
- Worker: A background process for slow jobs such as sending reminders, parsing PDFs, or creating embeddings.
- Queue: A waiting line for background jobs.
- RAG: Retrieval-Augmented Generation. The AI answers using approved internal knowledge instead of guessing.
- Embedding: A numeric representation of text used for semantic search.
- RBAC: Role-Based Access Control. Permissions based on roles such as Studio Director, Admin, Practitioner, Finance, and Marketing.
- RLS: Row-Level Security. Database-level policy that controls which rows a user can access.
- Audit log: A permanent record of sensitive actions, such as editing finance data or opening clinical case data.
- Hexagonal architecture: A pattern that separates business logic from external tools like WhatsApp, payment gateway, OpenAI, or Supabase.
- Modular monolith: One deployable app split into clean internal modules.
- Strangler migration: Rebuilding a system gradually instead of replacing everything at once.

3 Current System Audit

3.1 What Already Exists Based on Screenshots

The current internal system appears to contain:

- Strategic Overview dashboard.
- Monthly revenue and active client metrics.
- Practitioner performance and utilization metrics.
- AI lead conversion and token/cost visibility.
- Executive AI Business Agent with Supabase data integration.
- Financial strategy dashboard with ledger and report export.
- Client LTV and milestone dashboard.
- Chronic Case Registry.
- Team Attendance.
- Live Chat with AI agent and manual intervention.
- Appointments.
- Practitioner account management.

- Approvals and payroll workspace.

3.2 Strengths

1. The system is already business-specific, not generic.
2. AI is tied to business operations, not only chatbot gimmick.
3. The dashboard shows executive-level metrics.
4. There is evidence of approval workflow.
5. Client LTV and chronic cases create a strong vertical advantage.

3.3 Red Flags

1. Financial summary and ledger values must be reconciled carefully.
2. Some screens show loading states that may hide API errors or empty data.
3. Appointment core must be verified because it is the operational foundation.
4. AI appears connected to sensitive data, so governance must be strict.
5. User permissions need to be more granular than basic roles.

4 Product Requirements

5 01 - Product Requirements Document

6 HOM Studio OS v2

Generated: 2026-05-25

6.1 1. Product Summary

HOM Studio OS v2 is an internal operating system for a clinical pilates/physio studio. It helps the studio manage appointments, clients, practitioners, chronic cases, session notes, finance, payroll-related approvals, WhatsApp communication, AI-assisted business analysis, and knowledge management.

The product is not a generic fitness booking app. It is closer to a vertical operating system for a premium clinical wellness studio.

6.2 2. Product Vision

Create one trusted platform where the business owner, admin, practitioners, finance, and marketing team can operate the studio without scattered spreadsheets, manual WhatsApp tracking, unclear financial summaries, or uncontrolled AI behavior.

6.3 3. Main Problems

6.3.1 Problem 1 - Operational Data Is Spread Out

Appointments, WhatsApp messages, finance, commissions, client lifetime value, practitioner attendance, and clinical case data can easily become disconnected.

6.3.2 Problem 2 - Finance Needs One Source of Truth

Financial dashboards must never show conflicting totals. If a strategic dashboard and finance ledger show different numbers for the same period, the owner cannot trust the system.

6.3.3 Problem 3 - AI Must Be Useful but Controlled

The current idea includes AI, multi-LLM usage, and chatbot intelligence. That can improve productivity, but it is dangerous if AI can diagnose, approve payroll, change finance, or send campaigns without approval.

6.3.4 Problem 4 - Clinical Context Requires More Care Than Fitness Booking

Clinical cases can include chronic pain, post-surgery rehab, scoliosis, osteoarthritis, postnatal recovery, or other sensitive conditions. The system must protect this data and avoid AI overreach.

6.3.5 Problem 5 - Owner Needs No-Code Knowledge Control

The business owner should be able to upload PDF, DOCX, XLSX, image, SOP, pricing, and campaign documents, review extracted knowledge, test AI answers, publish changes, and rollback if needed.

6.4 4. Target Users

User Type	What They Need
Studio Director / Owner	Executive dashboard, finance, approvals, AI business insights, knowledge control
Admin / Front Desk	Appointment creation, reschedule, WhatsApp inbox, client support
Practitioner	Daily schedule, client context, chronic case summary, session notes
Finance Admin	Ledger, expenses, commissions, reimbursements, reports
Marketing Admin	WhatsApp blast drafts, segment insights, campaign approvals
AI Agent Service	Read-only tools, intent classification, drafting, summarization, behavior extraction

6.5 5. Product Goals

1. Make the studio operation easier to run daily.
2. Make finance calculations consistent and auditable.
3. Give the owner a trusted dashboard for decisions.
4. Make AI helpful but bounded by rules and approvals.
5. Let owner update AI knowledge without coding.
6. Learn customer behavior from chat interactions safely.
7. Keep architecture simple enough for a solo developer.
8. Keep deployment cost reasonable while allowing growth.

6.6 6. Non-Goals

The first rebuild should not include:

- Public marketplace for multiple studios.
- Native iOS/Android app.
- Full legal-grade medical record system.
- AI diagnosis.
- Autonomous refund, payroll approval, or clinical note editing.
- Microservices.
- Full accounting replacement.
- Full ERP.
- n8n as the core brain.

6.7 7. MVP Feature Scope

6.7.1 Phase 1 - Foundation

- Auth
- Role and permission system
- User management
- Client management
- Practitioner management
- Service catalog
- Appointment core
- Audit logs
- Base dashboard layout

6.7.2 Phase 2 - Clinical Operations

- Chronic case registry
- Practitioner assignment
- Session notes
- Note lock/unlock workflow
- Clinical access audit

6.7.3 Phase 3 - Finance

- Revenue ledger
- Expenses
- COGS
- Gross profit
- Operating expense
- Net profit
- Practitioner commission
- Reimbursement request
- PDF report export

6.7.4 Phase 4 - Communication

- WhatsApp inbox
- Manual intervention
- AI draft reply
- Message tagging
- Reminder jobs
- Blast approval workflow

6.7.5 Phase 5 - Knowledge Studio

- Upload PDF/DOCX/XLSX/image
- Extract content
- Owner review
- Knowledge scope assignment
- Chunking and embedding
- AI test lab
- Publish and rollback

6.7.6 Phase 6 - Behavior Intelligence

- Intent extraction from chat
- Topic clustering

- Customer behavior profile
- Unanswered question detection
- Campaign segment suggestion
- Knowledge update suggestions

6.7.7 Phase 7 - Executive Intelligence

- Strategic dashboard
- Client LTV leaderboard
- Milestone tiers
- Practitioner productivity
- AI cost dashboard
- Read-only AI Business Agent

6.8 8. Functional Requirements

6.8.1 8.1 Appointment Module

Must support:

- Create appointment.
- Reschedule appointment.
- Cancel appointment.
- Mark done.
- Mark no-show.
- Assign practitioner.
- Assign service type.
- Link appointment to client.
- Link appointment to package/payment when available.
- Store status history.
- Prevent overlapping appointment for the same practitioner.

Acceptance criteria:

- The same practitioner cannot have two appointments at the same time.
- Every reschedule stores old and new time.
- Every cancel stores reason and actor.
- Appointment changes create audit logs.

6.8.2 8.2 Clinical Case Registry

Must support:

- Client condition label.
- Status: active, monitoring, resolved, archived.
- Assigned practitioner.
- Severity or priority flag.
- Session count.
- Case summary.
- Restricted access by permission.

Acceptance criteria:

- Only permitted roles can view clinical cases.
- Every access to a clinical case is logged.
- AI cannot access raw clinical notes by default.

6.8.3 8.3 Session Notes

Must support:

- Draft note.
- Finalize note.
- Lock after finalization.
- Unlock request.
- Approve/reject unlock.
- Edit history.

Acceptance criteria:

- A finalized note cannot be edited directly.
- Unlock request needs reason.
- Approval action must be auditable.

6.8.4 8.4 Finance

Must support:

- Revenue entries.
- Expense entries.
- COGS classification.
- Operating expense classification.
- Commission calculation.
- Reimbursement claims.
- Monthly summary.
- PDF report export.

Acceptance criteria:

- Dashboard summary and ledger use the same period filter.
- No fake zero values if data failed to load.
- Every finance change is logged.

6.8.5 8.5 WhatsApp Inbox

Must support:

- Conversation list.
- Client linkage.
- AI draft reply.
- Human takeover.
- Conversation tags.
- Message status.
- Suggested action.

Acceptance criteria:

- AI cannot confirm reschedule without backend slot check.
- AI cannot diagnose.
- AI cannot promise refund.
- Human takeover disables auto-reply for that conversation.

6.8.6 8.6 Knowledge Studio

Must support:

- File upload.
- Document extraction.
- Spreadsheet mapping.
- Image/document OCR or description where appropriate.
- Owner review.
- Knowledge scope.

- Test lab.
- Publish/rollback.

Acceptance criteria:

- Uploaded knowledge is not active until published.
- Every answer in test lab shows source chunks.
- Owner can rollback to previous version.
- XLSX files are mapped structurally, not only chunked as text.

6.8.7 8.7 AI Business Agent

Must support:

- Read-only finance queries.
- Read-only LTV queries.
- Read-only commission summaries.
- Source-cited answers from internal data.
- Cost/token logs.
- Prompt/version tracking.

Acceptance criteria:

- AI Business Agent cannot mutate data in MVP.
- Every AI answer is logged with model, prompt version, and data sources.
- AI response must mention period/context if financial.

6.9 9. Non-Functional Requirements

6.9.1 Performance Targets

Area	MVP Target
Dashboard initial load	Under 3 seconds for normal data volume
Appointment create/update	p95 under 700 ms excluding network spikes
WhatsApp webhook handling	acknowledge quickly and process async
AI draft reply	under 8-15 seconds, fallback if timeout
Document ingestion	async; small documents under 5 minutes
Queue delay	under 10 seconds in normal operation

6.9.2 Security Requirements

- RBAC in backend.
- RLS in Supabase for sensitive data.
- Service role only server-side.
- Audit log for sensitive reads/writes.
- PII masking before AI calls.
- No hardcoded secrets.
- Environment variables for all provider keys.

6.9.3 Reliability Requirements

- Idempotent webhook handlers.
- Retry failed jobs.
- Dead-letter queue for failed jobs.
- Clear error states.
- Daily backups in production tier.

6.10 10. Success Metrics

Metric	Why It Matters
Appointment error rate	Core operational reliability
No-show rate	Business efficiency
Rebooking rate	Retention
Package renewal rate	Revenue health
LTV per client	Long-term business value
Practitioner utilization	Team productivity
AI cost per resolved conversation	AI efficiency
AI escalation rate	Safety and accuracy indicator
Financial report discrepancy count	Trust in dashboard
Knowledge test pass rate	AI answer quality

6.11 11. Major Risks

Risk	Impact	Mitigation
Big-bang rewrite breaks operations	High	Use phased migration
Finance calculations inconsistent	High	Single finance service, tests, audit logs
AI exposes sensitive data	High	Scope control, masking, logging
n8n logic becomes hidden dependency	Medium	Move core automation into code-first worker
UI looks good but data model weak	High	Build core schema first
Document ingestion poor quality	Medium	Owner review before publish
Multi-LLM increases cost	Medium	Model gateway, cost dashboard

6.12 12. Out of Scope Until Later

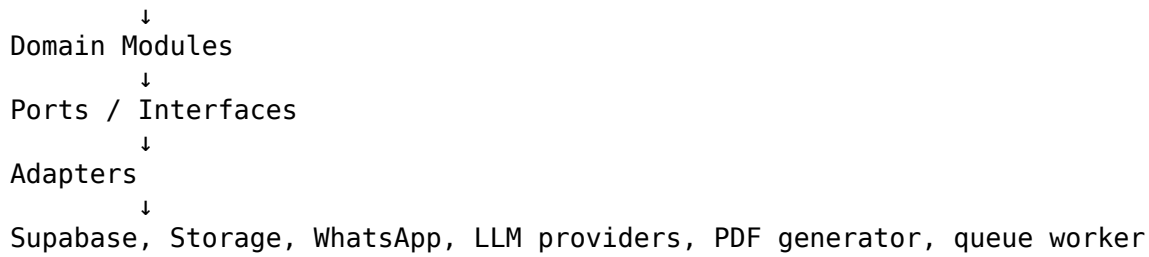
- Public mobile member app.
- Multi-branch accounting consolidation.
- Insurance/medical billing.
- Full HR payroll and tax.
- Automatic treatment recommendation.
- Real-time movement analysis from video.

7 System Blueprint

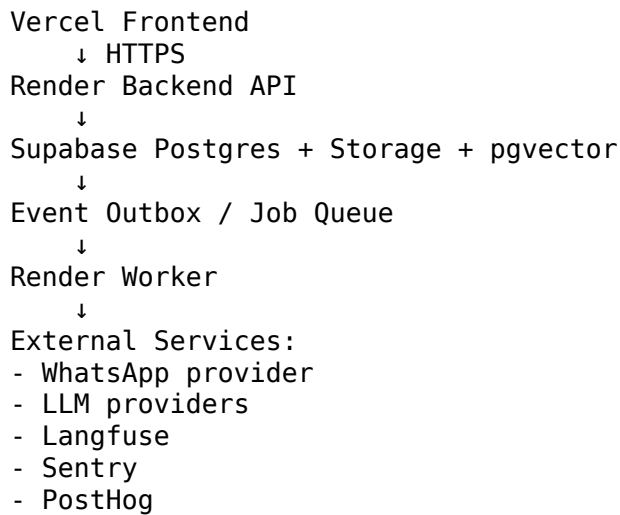
8 02 - System Blueprint

8.1 1. High-Level Architecture

Business Owner / Admin / Practitioner
↓
Web Dashboard - Next.js / React
↓
Backend API / BFF
↓
Application Use Cases



8.2 2. Runtime Architecture



8.3 3. Core Modules

auth/
roles/
clients/
practitioners/
services/
appointments/
clinical-cases/
session-notes/
finance/
commissions/
reimbursements/
whatsapp/
knowledge-studio/
behavior-intelligence/
ai-gateway/
approvals/
reports/
audit/

8.4 4. Module Responsibilities

8.4.1 auth/

Handles login, session, identity, and user profile. It must not contain business logic.

8.4.2 roles/

Handles roles and permissions. Every sensitive API endpoint must check permissions here.

8.4.3 clients/

Stores client profiles, contact info, lifecycle status, and basic non-sensitive metadata.

8.4.4 practitioners/

Stores practitioner account, display info, availability, attendance, and relation to appointments.

8.4.5 appointments/

The operational core. Responsible for booking, reschedule, cancel, done, no-show, status history, and practitioner availability checks.

8.4.6 clinical-cases/

Stores restricted clinical case summaries and assignment to practitioners.

8.4.7 session-notes/

Stores session notes with lock/unlock workflow.

8.4.8 finance/

Stores ledger entries, period summaries, gross/net calculations, expenses, and report data.

8.4.9 commissions/

Calculates practitioner commission based on approved business rules.

8.4.10 reimbursements/

Handles team reimbursement claims and approval flow.

8.4.11 whatsapp/

Handles conversation inbox, messages, AI draft, human takeover, and blast approval.

8.4.12 knowledge-studio/

Handles document upload, extraction, review, chunking, embedding, test lab, publish, and rollback.

8.4.13 behavior-intelligence/

Extracts structured signals from chats: intent, objections, schedule preference, churn risk, unanswered questions, and segment ideas.

8.4.14 ai-gateway/

Single gateway for all LLM calls. No feature should call OpenAI/Anthropic/Gemini directly.

8.4.15 approvals/

Generic approval engine for reimbursement, note unlock, WhatsApp blast, and sensitive AI action requests.

8.4.16 audit/

Records sensitive reads and writes.

8.5 5. Data Flow - Appointment Creation

Admin creates appointment

↓

API validates payload with Zod

↓

Permission check: `can_manage_appointments`

↓

Appointment use case checks practitioner availability

↓

Database transaction creates appointment + status history

↓

Audit log created

↓

Event outbox creates `AppointmentCreated` event

↓

Worker sends reminder or confirmation message

8.6 6. Data Flow - WhatsApp AI Draft

Incoming WhatsApp message

↓

Webhook acknowledges quickly

↓

Message saved

↓

Event outbox creates `ConversationMessageReceived`

↓

Worker calls AI Gateway

↓

AI Gateway retrieves allowed knowledge scope

↓

Policy guard checks risk

↓

Draft response saved

↓

If safe and auto-reply allowed: send

↓

If sensitive: request human intervention

8.7 7. Data Flow - Knowledge Upload

Owner uploads file

↓

File stored in Supabase Storage

↓

`knowledge_sources` row created with status uploaded

↓
Worker extracts text/tables/images
↓
Owner reviews extraction
↓
Owner assigns scope
↓
Worker chunks and embeds approved content
↓
Owner runs test cases
↓
Owner publishes version

8.8 8. Data Flow - Behavior Learning

Conversation closed or updated
↓
Worker extracts intent/topic/objection/sentiment
↓
Structured behavior events saved
↓
Weekly aggregation job creates insights
↓
Owner sees recommended FAQ/rule/campaign update
↓
Owner approves, edits, or rejects
↓
Approved update goes to Knowledge Studio draft

8.9 9. Data Flow - Finance Report

Finance admin enters ledger data
↓
Finance use case validates category and period
↓
Ledger entry saved
↓
Monthly summary recalculated
↓
Report snapshot created
↓
Owner can export PDF

8.10 10. Service Boundaries

8.10.1 Core transaction boundaries

The following must be backend-controlled only:

- Appointments
- Finance
- Commissions
- Reimbursements
- Clinical cases
- Session notes
- Payment/package state if added later
- User permissions

8.10.2 AI boundaries

AI may draft, classify, summarize, and suggest. Backend decides whether actions are allowed.

8.10.3 Worker boundaries

Worker may process async jobs. Worker must call domain/use-case functions, not bypass rules.

8.11 11. Integration Boundaries

Integration	Role	Rule
WhatsApp provider	Communication channel	Never source of truth
LLM providers	Drafting/classification/analysis	No direct mutation
Supabase	Database/auth/storage/vector	Source of truth
Langfuse	AI observability	No business data beyond logged traces where allowed
PostHog	Product analytics	Avoid clinical/PII event payloads
Sentry	Error tracking	Scrub PII

8.12 12. Scalability Model

Start simple:

- 1 backend instance
- 1 worker instance
- 1 database
- 1 queue/cache

Scale by:

1. Optimize database queries.
2. Add caching for dashboard cards.
3. Add worker concurrency.
4. Split heavy document worker from general worker.
5. Upgrade Supabase compute.
6. Add VPS only for heavy AI/document ingestion if needed.

9 Architecture Decisions

10 03 - Architecture Decisions

10.1 ADR 001 - Use Modular Monolith, Not Microservices

10.1.1 Decision

Use a modular monolith.

10.1.2 Reason

A solo developer needs low operational complexity. Microservices add deployment complexity, network failures, distributed transactions, observability burden, and debugging difficulty.

10.1.3 Consequence

All modules live in one repo and can be deployed together. Internal boundaries must be enforced through folder structure, interfaces, and tests.

10.2 ADR 002 - Use Selective Hexagonal Architecture

10.2.1 Decision

Use hexagonal architecture only for high-risk modules:

- Appointments
- Finance
- Clinical notes
- WhatsApp
- AI Gateway
- Knowledge Studio
- Approvals

Use normal layered CRUD for simple settings and static catalogs.

10.2.2 Reason

Full clean architecture everywhere is too heavy. But direct coupling to external tools is dangerous in finance, AI, and communication.

10.2.3 Consequence

Each high-risk module must separate:

Domain logic
Application use case
Ports/interfaces
Adapters/infrastructure

10.3 ADR 003 - Remove n8n from Critical Path

10.3.1 Decision

Do not use n8n as the main intelligence layer in v2. Use code-first workers and an AI Gateway.

10.3.2 Reason

n8n is flexible but difficult to test and version as core business logic. Core logic must live in the repo.

10.3.3 Consequence

n8n may remain temporarily for migration or non-critical workflows, but the target system should not depend on n8n for appointment, finance, payroll, clinical notes, or AI safety.

10.4 ADR 004 - Use Event Outbox and Worker

10.4.1 Decision

Use an event outbox table and a worker queue. Start with pg-boss or a Postgres-backed queue. Add Redis/BullMQ only when needed.

10.4.2 Reason

Document ingestion, WhatsApp messages, AI calls, and report generation should not block API requests.

10.4.3 Consequence

All slow tasks become jobs with retry, status, and failure logs.

10.5 ADR 005 - Use Multi-LLM Through a Gateway Only

10.5.1 Decision

All LLM calls must go through ai-gateway.

10.5.2 Reason

Without a gateway, costs, prompts, model choice, logs, and safety rules will spread everywhere.

10.5.3 Consequence

Features call business functions like:

```
classifyIntent()  
draftReply()  
summarizeConversation()  
extractBehaviorSignals()  
analyzeFinance()  
parseDocument()
```

They do not call provider APIs directly.

10.6 ADR 006 - AI Must Be Human-in-the-Loop for Sensitive Actions

10.6.1 Decision

AI cannot autonomously approve or modify:

- Finance
- Payroll/commission
- Reimbursement
- Clinical note unlock
- Refunds
- Diagnosis
- WhatsApp blast to many customers

10.6.2 Reason

The system touches sensitive operational, financial, and clinical-adjacent data.

10.6.3 Consequence

Sensitive AI suggestions become approval_requests.

10.7 ADR 007 - Use Knowledge Studio Instead of Hardcoded Prompts

10.7.1 Decision

Create a dashboard where owner can upload documents, review extracted knowledge, test AI responses, publish, and rollback.

10.7.2 Reason

The business owner needs to update pricing, SOP, FAQ, campaign tone, and service knowledge without developer involvement.

10.7.3 Consequence

Knowledge has versions, scopes, review status, and test runs.

10.8 ADR 008 - Deployment Stack

10.8.1 Decision

Phase 1 production:

Vercel frontend
Render backend
Render worker
Render Key Value or Postgres queue
Supabase database/storage/vector

10.8.2 Reason

This balances cost, performance, Singapore-region availability, and low DevOps burden.

10.8.3 Consequence

Avoid Heroku for primary deployment because Common Runtime region is US/EU only. Avoid full VPS until heavy workers justify it.

10.9 ADR 009 - Frontend Quality Stack

10.9.1 Decision

Use:

Next.js
React 19
TypeScript
Tailwind
shadcn/ui
Radix UI
Motion.dev
21st.dev Agent Elements or compatible AI UI components
Storybook
Playwright

UIPro can be used only after Figma is clean.

10.9.2 Reason

The platform needs a premium, stable, dashboard-heavy frontend.

10.9.3 Consequence

Generated UI must be refactored into reusable components and pass quality checks.

10.10 ADR 010 - Source of Truth

10.10.1 Decision

Supabase/Postgres is the source of truth.

10.10.2 Reason

WhatsApp, AI, workers, files, and dashboards are secondary. The database must be consistent.

10.10.3 Consequence

No external tool can directly decide core state without backend validation.

11 Database Blueprint

The complete detailed schema is in 04_DATABASE_SCHEMA.md. This section summarizes the most important table groups.

11.1 Identity and Permissions

users
roles
permissions
user_roles
role_permissions

11.2 Operations

clients
client_profiles
practitioners
services
appointments
appointment_status_history
practitioner_attendance

11.3 Clinical Workflow

client_conditions
chronic_cases
session_notes
clinical_note_unlock_requests

11.4 Finance

financial_ledger
monthly_summary
therapist_commissions
reimbursements

11.5 Communication and AI

whatsapp_conversations
whatsapp_messages
whatsapp_blasts
knowledge_sources
knowledge_extractions
knowledge_chunks
ai_agent_logs
conversation_events
customer_behavior_profiles

11.6 Worker and Governance

event_outbox
audit_logs
approval_requests
report_exports

12 AI Knowledge Studio

13 06 - AI Knowledge Studio and Behavior Intelligence

13.1 1. Goal

Knowledge Studio lets the business owner update the AI knowledge base without coding. It supports uploaded files such as PDF, DOCX, XLSX, CSV, images, screenshots, SOP documents, pricing sheets, and campaign examples.

The feature also learns customer behavior from WhatsApp conversations, but it does not automatically train the model or change policy without human approval.

13.2 2. Why This Matters

A studio owner often changes:

- Pricing
- Service packages
- Campaign messages
- Cancellation rules
- Practitioner schedule info
- FAQs
- Tone of customer service
- Safety disclaimers
- Admin SOP

Hardcoding all of this into prompts will become messy. Knowledge Studio turns business knowledge into versioned, reviewable, testable data.

13.3 3. Main Pages

13.3.1 3.1 Knowledge Sources

Shows all uploaded files.

Columns:

- Title

- Type
- Scope
- Status
- Version
- Uploaded by
- Last processed
- Published status
- Actions

Statuses:

uploaded
processing
extracted
review_needed
approved
embedded
tested
published
archived
failed

13.3.2 3.2 Upload Panel

The owner can upload:

- PDF
- DOCX
- XLSX
- CSV
- PNG/JPG
- Screenshots

The upload form asks:

- Title
- Document type
- Scope
- Is this public chatbot knowledge?
- Is this internal admin knowledge?
- Is this finance knowledge?
- Is this clinical safety knowledge?

13.3.3 3.3 Extraction Review

After processing, the owner sees:

- Extracted text
- Detected tables
- Image descriptions
- Possible errors
- Confidence score
- Suggested scope
- Suggested chunks

The owner can edit before publishing.

13.3.4 3.4 Spreadsheet Mapping

XLSX files are not treated like normal text.

The owner must select:

- Sheet name
- Header row
- Date column
- Amount column
- Category column
- Practitioner column if relevant
- Client column if relevant
- Whether this sheet is finance, attendance, clients, services, or pricing

13.3.5 3.5 Business Rules Editor

Allows owner to create rules like:

If customer asks about pain or injury:

- Do not diagnose.
- Explain that assessment is recommended.
- Escalate to human if post-surgery or severe symptoms are mentioned.

Rules have:

- Scope
- Priority
- Active/inactive
- Version
- Test cases

13.3.6 3.6 Chatbot Behavior Profile

Owner can set:

- Language: Indonesian, English, mixed
- Tone: warm, premium, concise, clinical-safe
- Emoji usage: none, minimal, allowed
- Autonomy level: draft only, safe FAQ auto-reply, booking info auto-reply
- Escalation rules

13.3.7 3.7 Test Lab

Owner can test AI before publishing.

Example test questions:

Berapa harga private session?

Saya habis operasi lutut, boleh ikut kelas apa?

Saya mau refund.

Saya mau reschedule dengan Firly hari Kamis.

Apa bedanya clinical pilates dan pilates biasa?

Output:

- AI answer
- Retrieved sources
- Confidence
- Policy flags
- Latency
- Cost estimate
- Pass/fail

13.3.8 3.8 Publish and Rollback

Knowledge is not active until published.

Publish record must save:

- Version
- Published by
- Published at
- Affected scope
- Test result
- Rollback target

13.4 4. Knowledge Scopes

Scope	Used By	Examples
public_chatbot	WhatsApp customer AI	FAQ, location, service explanation, pricing
internal_admin	Admin assistant	SOP, reschedule rules, refund process
clinical_safety	Policy guard	non-diagnostic rules, escalation triggers
finance	AI Business Agent	category definitions, commission rules
marketing	Campaign assistant	tone, campaign examples, segment logic
owner_only	Studio Director	sensitive strategy notes

13.5 5. Document Ingestion Pipeline

```
Upload
↓
Store raw file
↓
Create knowledge_sources row
↓
Extract content
↓
Owner review
↓
Chunk content
↓
Create embeddings
↓
Run test cases
↓
Publish
```

13.6 6. Parser Recommendation

Start simple:

- PDF/DOCX: Docling or Unstructured.
- XLSX: custom spreadsheet mapper.
- Images: OCR/vision only if needed.
- CSV: direct structured import.

Do not use expensive vision parsing for every file by default. Use it only when normal extraction fails or the file is image-heavy.

13.7 7. RAG Retrieval Rules

When AI answers:

1. Determine scope.
2. Retrieve only allowed knowledge chunks.
3. Exclude archived/draft knowledge.
4. Include business rules for that scope.
5. Generate answer.
6. Run policy guard.
7. Return answer and sources.

13.8 8. AI Safety Rules

AI must not:

- Diagnose medical conditions.
- Promise healing.
- Promise refund.
- Create discount not approved by owner.
- Confirm reschedule without backend check.
- Read finance data if user lacks permission.
- Read clinical notes unless specifically allowed and masked.

13.9 9. Behavior Intelligence

13.9.1 9.1 What to Extract from Chat

From WhatsApp conversations, extract structured data:

intent
topic
objection
sentiment
urgency
preferred practitioner
preferred day/time
service interest
price sensitivity
reschedule reason
churn risk
unanswered question
conversion outcome

13.9.2 9.2 Example

Chat:

Hi, I have a session with Firly on Thursday. Can I reschedule?

Extraction:

```
{  
  "intent": "reschedule_request",  
  "preferredPractitioner": "Firly",  
  "urgency": "medium",  
}
```

```

    "actionNeeded": "check_available_slots",
    "risk": "appointment_change"
}

```

13.9.3 9.3 Behavior Dashboard

Owner should see:

- Top questions this week
- Top unanswered questions
- Most common reschedule reasons
- Lead objections
- Campaign response trends
- Practitioner demand
- Price sensitivity trends
- FAQ gaps
- Suggested knowledge updates

13.10 10. Human Approval Loop

AI discovers pattern

↓

AI suggests rule/FAQ/campaign update

↓

Owner reviews

↓

Owner edits or rejects

↓

If approved, create Knowledge Studio draft

↓

Run test lab

↓

Publish

13.11 11. Multi-LLM Gateway

Use model aliases, not hardcoded providers.

Example:

```

fast_classification_model
premium_reasoning_model
vision_document_model
embedding_model
critic_model

```

Routing:

- Intent classification: fast_classification_model
- WhatsApp reply draft: balanced_chat_model
- Finance analysis: premium_reasoning_model
- Document image parsing: vision_document_model
- RAG evaluation: critic_model
- Embeddings: embedding_model

13.12 12. Observability

Log for every AI call:

- Feature
- Model alias
- Provider
- Prompt version
- Knowledge version
- Retrieved chunks
- Latency
- Token usage
- Cost
- Result status
- Human override

13.13 13. MVP Build Order

1. File upload.
2. Knowledge source table.
3. PDF/DOCX text extraction.
4. Owner review screen.
5. Manual chunking or simple chunking.
6. Embeddings.
7. Retrieval.
8. Test Lab.
9. Publish/rollback.
10. Behavior extraction.
11. Weekly insights.
12. Suggested knowledge updates.

14 Worker and Automation Plan

15 07 - Worker, Queue, and Automation Plan

15.1 1. Goal

Move slow or unreliable tasks out of the API request path.

The API should respond quickly. Workers handle background work.

15.2 2. Tasks That Must Run in Worker

- WhatsApp message processing
- AI draft generation
- Document extraction
- Spreadsheet parsing
- Embedding generation
- RAG evaluation
- Behavior extraction
- Weekly insight generation
- PDF report generation
- Reminder sending
- Campaign sending

15.3 3. Recommended MVP Queue

Use one of these:

Option 1: Postgres event_outbox + pg-boss

Option 2: BullMQ + Redis Key Value/Redis

For simplest start with Supabase/Postgres:

```
event_outbox table
worker polls pending events
worker processes jobs
worker updates status
failed jobs retry
```

15.4 4. Event Types

```
AppointmentCreated
AppointmentRescheduled
AppointmentCancelled
AppointmentCompleted
InboundWhatsAppMessageReceived
ConversationClosed
KnowledgeSourceUploaded
KnowledgeSourceApproved
KnowledgeSourcePublished
FinanceLedgerChanged
ReimbursementRequested
ClinicalNoteUnlockRequested
WhatsAppBlastApproved
```

15.5 5. Worker Rules

- Worker must never bypass domain rules.
- Worker must use the same use cases as the API.
- Worker must be idempotent.
- Worker must log failures.
- Worker must avoid infinite retries.
- Worker must attach correlation_id to every job.

15.6 6. Retry Policy

Suggested defaults:

```
attempt 1: immediately
attempt 2: after 1 minute
attempt 3: after 5 minutes
attempt 4: after 30 minutes
attempt 5: mark failed and alert
```

15.7 7. Dead-Letter Queue

If job fails permanently:

- Mark as failed.
- Store error message.
- Store last payload.
- Alert admin/developer.
- Provide retry button in internal admin page.

15.8 8. Scheduled Jobs

Daily:

- Reminders for upcoming appointments.
- Clean old temp files.
- Recalculate active client metrics.

Weekly:

- Behavior insight summary.
- Top unanswered questions.
- AI cost summary.
- Practitioner utilization summary.

Monthly:

- Finance monthly summary snapshot.
- LTV leaderboard snapshot.
- Report export job.

15.9 9. Workflow Without n8n

Old approach:

Webhook → n8n → AI logic → action

New approach:

Webhook → Backend → Event Outbox → Worker → AI Gateway → Backend rule check → action/approval

15.10 10. Optional n8n Migration Bridge

If the old system still needs n8n temporarily:

- n8n can call restricted backend endpoints.
- n8n cannot write directly to database.
- Every n8n call needs API key, correlation_id, and scope.
- n8n output must be treated as suggestion, not truth.

15.11 11. Worker Dashboard

Create internal page:

/jobs

Show:

- Pending jobs
- Running jobs
- Failed jobs
- Retry button
- Job payload summary
- Last error
- Duration
- Attempts

15.12 12. Performance Benchmarks

Job Type	MVP Target
WhatsApp message saved	under 1 second
AI draft generated	under 8-15 seconds
Small PDF extraction	under 5 minutes
Embedding batch	async, no API blocking
Monthly report PDF	async if over 5 seconds
Queue delay normal	under 10 seconds

15.13 13. Scaling Path

Phase 1:

one worker handles all jobs

Phase 2:

general worker + AI/document worker

Phase 3:

API remains on Render

heavy document worker on VPS

Phase 4:

separate queues per job type

add concurrency controls

16 Frontend UI/UX Plan

17 08 - Frontend UI/UX Guide

17.1 1. Goal

Create a premium, calm, trustworthy dashboard for a clinical studio. The UI should feel modern but not playful. Finance, clinical cases, and payroll screens must feel serious and readable.

17.2 2. Recommended Stack

Next.js
 React 19
 TypeScript
 Tailwind CSS
 shadcn/ui
 Radix UI
 Motion.dev
 21st.dev Agent Elements or equivalent
 Storybook
 Playwright
 Lighthouse CI

17.3 3. Motion.dev Usage

Use Motion.dev for subtle polish:

- Page transitions
- Card entrance

- Modal open/close
- Chat message arrival
- File upload progress
- Approval status movement
- Loading states

Do not over-animate finance and clinical screens.

Motion presets:

critical screens: 120-180ms, fade only

normal dashboard cards: 180-250ms, fade + small y offset

chat/AI: 200-300ms, soft slide

empty states: can be slightly more expressive

Always support reduced motion.

17.4 4. UIPro Usage

UIPro is useful only after the Figma file is clean.

Before using UIPro:

- Use Auto Layout.
- Name layers clearly.
- Create variants for buttons/cards/tables.
- Use consistent spacing.
- Use design tokens.
- Define responsive behavior.

After UIPro generates code:

1. Do not commit blindly.
2. Refactor into reusable components.
3. Replace hardcoded values with tokens.
4. Add TypeScript props.
5. Add Storybook story.
6. Add loading/empty/error states.

17.5 5. 21st.dev Usage

Use 21st.dev for inspiration and AI UI components.

Good use cases:

- AI Business Agent chat
- Knowledge Studio test lab
- File attachment UI
- Tool cards
- Model picker
- Command palette
- Empty states
- Approval cards

Treat generated/community code as external code. Review dependencies and security before use.

17.6 6. Design System

17.6.1 Colors

Use calm premium palette:

background: warm off-white / beige
sidebar: dark brown / charcoal
primary: muted gold or warm tan
success: calm green
warning: amber
error: muted red
text: dark neutral

17.6.2 Typography

- Headings: clear and calm.
- Tables: readable at small sizes.
- Finance numbers: tabular numeric style if possible.

17.6.3 Spacing

Use consistent spacing scale:

4, 8, 12, 16, 24, 32, 48

17.6.4 Radius and Shadows

- Cards: medium-large radius.
- Shadows: soft, not heavy.
- Tables: minimal borders, clear row hover.

17.7 7. Main Pages

17.7.1 7.1 Dashboard Layout

Required:

- Sidebar
- Top bar
- Page title
- Breadcrumb optional
- Global search optional
- User menu
- Content grid

17.7.2 7.2 Strategic Overview

Cards:

- Monthly revenue
- Active clients
- Practitioner utilization
- AI lead conversion
- Performance bonus pool
- AI cost

Charts:

- Revenue trend
- Client count trend
- Practitioner performance
- Studio utilization

17.7.3 7.3 Appointments

Views:

- Calendar
- Table
- Practitioner filter
- Status filter
- New appointment modal
- Reschedule modal

States:

- Loading calendar
- Empty day
- API error
- Conflict error

17.7.4 7.4 Live Chat

Layout:

- Conversation list
- Message thread
- Client context side panel
- AI draft box
- Manual intervention toggle

17.7.5 7.5 Knowledge Studio

Sections:

- Upload files
- Sources table
- Extraction review
- Spreadsheet mapping
- Test lab
- Publish/rollback

17.7.6 7.6 Finance

Sections:

- Period picker
- Summary cards
- Ledger table
- Expenses
- Commission
- Reimbursements
- PDF export

Rule: finance summary must never show fake zero if loading failed.

17.7.7 7.7 Clinical Cases

Sections:

- Restricted access banner
- Case list
- Client summary
- Assigned practitioner

- Status
- Session count

17.7.8 7.8 Approvals

Kanban or table:

- Reimbursements
- Note unlock requests
- WhatsApp blast approvals
- Processed history

17.8 8. Component Library

Suggested components:

KpiCard
 RevenueChart
 PractitionerMetricCard
 AppointmentCalendar
 AppointmentStatusBadge
 ClientProfileCard
 ClinicalRiskBadge
 FinanceLedgerTable
 ApprovalRequestCard
 ChatConversationList
 ChatMessageBubble
 AIDraftPanel
 KnowledgeSourceCard
 FileUploadDropzone
 ProcessingTimeline
 TestLabResult
 AuditLogTable

17.9 9. Accessibility Checklist

Every screen must support:

- Keyboard navigation.
- Visible focus state.
- Proper labels for forms.
- Sufficient color contrast.
- No meaning by color alone.
- Reduced motion.
- Readable error messages.

17.10 10. Frontend Quality Gate

Before merge:

- TypeScript passes.
- Lint passes.
- Storybook renders components.
- Playwright smoke test passes.
- Lighthouse performance not worse than threshold.
- No hardcoded fake data in final screens.
- Loading/empty/error states exist.

18 Deployment and Capacity Plan

19 09 - Deployment Phases and Capacity Plan

19.1 1. Final Recommendation

Use this for MVP production:

Vercel: frontend
Render: backend API
Render: background worker
Render Key Value or Postgres queue: queue/cache
Supabase: database, auth, storage, pgvector
Primary region: Singapore where possible

Avoid Heroku as primary backend if most users are in Indonesia/Singapore because Heroku Common Runtime is US/EU only. Avoid full VPS at the beginning unless heavy workers justify it.

19.2 2. Phase 0 - Development Prototype

19.2.1 Use When

- Developer only.
- Dummy data.
- No real finance/clinical data.
- Less than 100 clients.

19.2.2 Stack

Vercel Hobby
Supabase Free
Render Starter backend or local backend
No paid Redis unless needed

19.2.3 Benchmarks

Dashboard load: under 4 seconds
API CRUD p95: under 800 ms
AI response: async, under 20 seconds acceptable

19.2.4 Cost Range

Approx. \$0-\$25/month

19.3 3. Phase 1 - MVP Production Internal

19.3.1 Use When

- 1 studio.
- 3-10 staff/practitioners.
- 300-1,000 clients.
- 500-2,000 appointments/month.
- 1,000-5,000 WhatsApp messages/month.
- 10-100 document uploads/month.

19.3.2 Stack

Vercel Pro
Supabase Pro - Singapore
Render backend Starter/Standard - Singapore
Render worker Starter
Render Key Value Starter or Postgres queue

19.3.3 Benchmarks

Dashboard initial load: under 3 seconds
Appointment create/update p95: under 700 ms
Chat backend p95: under 1 second, excluding provider delay
AI draft: under 8-15 seconds
Small document ingestion: async, under 5 minutes
Queue delay: under 10 seconds normal

19.3.4 Cost Range

Approx. \$69-\$120/month before LLM usage

19.4 4. Phase 2 - Stable Production

19.4.1 Upgrade Triggers

- Backend CPU often over 70%.
- Worker queue delay over 60 seconds.
- Appointment API p95 over 1 second.
- Dashboard frequently slow.
- PDF/XLSX ingestion blocks other jobs.
- Supabase database nearing plan limits.

19.4.2 Stack

Vercel Pro
Supabase Pro with compute upgrade if needed
Render backend Standard
Render worker Standard
Render Key Value Standard
Optional second worker for AI/document jobs

19.4.3 Benchmarks

API p95: under 500-800 ms
Dashboard cached cards: under 1.5 seconds
Queue delay: under 30 seconds
AI fallback if timeout
Error rate: under 1%

19.4.4 Cost Range

Approx. \$120-\$250/month before LLM usage

19.5 5. Phase 3 - Heavy AI and Document Processing

19.5.1 Upgrade Triggers

- Hundreds of PDF/XLSX/image files per month.
- OCR or vision parsing becomes slow.
- Embedding batch jobs create long queue backlog.
- Render worker becomes expensive.

19.5.2 Stack

Vercel frontend
Render API
Supabase database/storage/vector
VPS heavy worker only
Queue shared through Postgres/Redis

19.5.3 Benchmarks

API unaffected during ingestion batch
Normal backlog clears under 15 minutes
Heavy worker restarts automatically
Failed jobs enter retry/dead-letter queue

19.5.4 Cost Range

Approx. \$150-\$350/month before LLM usage

19.6 6. Phase 4 - Multi-Studio / SaaS Serious

19.6.1 Upgrade Triggers

- More than 5 studios.
- More than 10,000 clients.
- More than 20,000 appointments/month.
- More than 50,000 WhatsApp messages/month.
- Multi-tenant billing needed.
- SLA becomes important.

19.6.2 Stack Options

Render Pro/Scale
or VPS cluster
or Fly.io/Railway/D0 App Platform
or Kubernetes only if team/scale justifies it

19.6.3 Benchmarks

API p95: under 500 ms for core operations
Queue delay: under 10-30 seconds
DB query p95: under 100-200 ms for core tables
Error rate: under 0.5%
Backup restore tested monthly

19.6.4 Cost Range

Approx. \$350-\$1,000+/month before LLM usage

19.7 7. Things People Forget

19.7.1 7.1 AI Cost Can Exceed Hosting

Track:

- Cost per feature.
- Cost per conversation.
- Cost per document ingestion.
- Cost per model.
- Failed AI calls.

19.7.2 7.2 Region Alignment Matters

Try to keep backend, Redis/queue, and Supabase in Singapore region.

Bad:

Backend US/EU + Database Singapore

Better:

Backend Singapore + Database Singapore

19.7.3 7.3 File Storage and Egress

Knowledge Studio stores documents and extracted data. Add file size limits and retention policy.

19.7.4 7.4 Observability From Day One

Use at least:

Sentry

Langfuse

PostHog

basic uptime monitor

19.7.5 7.5 Backups and Restore

A backup that has never been restored is only hope, not a recovery plan.

19.8 8. Final Hosting Decision

Best initial path:

Vercel + Render + Supabase

Not recommended as primary:

Single Heroku app + Supabase

Use VPS later for:

heavy document parsing

OCR

batch embedding

AI evaluation

20 Security and Governance

21 10 - Security and Governance

21.1 1. Why Security Matters Here

This system handles:

- Client contact details
- WhatsApp conversations
- Clinical-adjacent case labels
- Session notes
- Finance and revenue
- Practitioner commission
- Reimbursement requests
- AI logs
- Uploaded files

So it needs stronger governance than a normal booking dashboard.

21.2 2. Role-Based Access Control

Roles:

super_admin
studio_director
admin_frontdesk
practitioner
finance_admin
marketing_admin
viewer
ai_agent_service

Permissions:

can_view_financials
can_edit_financials
can_export_financial_report
can_view_clients
can_manage_clients
can_view_clinical_cases
can_manage_clinical_cases
can_view_session_notes
can_edit_session_notes
can_request_note_unlock
can_approve_note_unlock
can_manage_appointments
can_reschedule_appointments
can_manage_practitioners
can_view_team_attendance
can_approve_reimbursements
can_approve_whatsapp_blast
can_send_whatsapp_message
can_use_ai_business_agent
can_view_ai_logs
can_manage_knowledge
can_publish_knowledge

21.3 3. Sensitive Actions Requiring Audit Log

Always log:

- View clinical case detail.
- View session note.
- Edit session note.
- Request note unlock.
- Approve/reject note unlock.
- Create/edit finance ledger.
- Recalculate monthly summary.
- Export finance report.
- Approve reimbursement.
- Approve WhatsApp blast.
- Send manual WhatsApp message.
- Use AI Business Agent for finance.
- Publish knowledge version.
- Change permissions.

21.4 4. AI Governance

AI may:

- Draft replies.
- Classify intent.
- Summarize conversations.
- Suggest knowledge updates.
- Analyze approved finance summaries.
- Extract behavior signals.

AI must not:

- Diagnose medical conditions.
- Prescribe treatment.
- Promise healing.
- Approve payroll/commission.
- Approve reimbursement.
- Unlock clinical notes.
- Send mass WhatsApp blast without approval.
- Modify finance.
- Confirm refund.
- Confirm reschedule without backend availability check.

21.5 5. PII and Data Masking

Before sending data to LLM:

- Remove phone numbers unless needed.
- Remove email unless needed.
- Use client ID or first name only where possible.
- Summarize clinical notes instead of sending raw notes.
- Avoid sending finance details to low-cost/general models.

21.6 6. Knowledge Scope Safety

Each knowledge document must have scope:

public_chatbot
internal_admin

clinical_safety
finance
marketing
owner_only

AI should retrieve only from allowed scope.

21.7 7. Supabase RLS

Enable RLS on sensitive tables.

Important reminder:

RLS is defense-in-depth.

Backend permission checks are still required.

21.8 8. Service Role Safety

Never expose Supabase service role key to frontend.

Allowed:

server-side backend
worker environment

Forbidden:

browser
public environment variables
client components

21.9 9. Secrets Management

Use environment variables:

SUPABASE_URL
SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
OPENAI_API_KEY
ANTHROPIC_API_KEY
GEMINI_API_KEY
WHATSAPP_ACCESS_TOKEN
LANGFUSE_PUBLIC_KEY
LANGFUSE_SECRET_KEY
SENTRY_DSN
POSTHOG_KEY

Never commit .env files.

21.10 10. Logs and Error Tracking

Scrub:

- Phone numbers
- Emails
- Full clinical notes
- Payment information
- Raw uploaded file contents
- API keys

21.11 11. Approval Workflow

Sensitive actions should create an approval request:

requested_by
request_type
target_type
target_id
payload
risk_level
status
reviewed_by
reviewed_at
review_notes

21.12 12. Minimum Production Checklist

- RBAC works.
- RLS enabled on sensitive tables.
- Audit logs created.
- Secrets are not exposed.
- Sentry installed.
- Langfuse installed for AI traces.
- Backups enabled.
- Restore process documented.
- Admin-only routes protected.
- AI does not mutate sensitive state.

22 Testing and Quality Gates

23 11 - Testing and Quality Gates

23.1 1. Goal

Make the system safe to change. Codex can generate code quickly, but you need tests and quality gates so it does not silently break appointment, finance, or AI governance.

23.2 2. Test Types

23.2.1 Unit Tests

Use for pure business rules:

- Appointment overlap check
- Commission calculation
- Finance summary calculation
- Permission checks
- AI policy guard
- Knowledge scope filtering

23.2.2 Integration Tests

Use for API + database behavior:

- Create appointment
- Reschedule appointment
- Create finance ledger entry

- Publish knowledge source
- Process webhook
- Create audit log

23.2.3 End-to-End Tests

Use Playwright:

- Login
- View dashboard
- Create appointment
- Reschedule appointment
- Upload knowledge document
- Run test lab
- Approve request

23.2.4 Load Tests

Use k6 for:

- Appointment API
- Dashboard API
- WhatsApp webhook
- Knowledge query

23.2.5 Visual Tests

Use Storybook/Chromatic or screenshot tests for:

- KPI cards
- Tables
- Modals
- Chat UI
- Knowledge Studio

23.3 3. Business Rule Tests

23.3.1 Appointment Overlap

Test cases:

- Same practitioner, overlapping time: reject.
- Same practitioner, adjacent time: allow.
- Different practitioner, same time: allow.
- End time before start time: reject.

23.3.2 Finance Summary

Test cases:

- Income increases revenue.
- COGS decreases gross profit.
- OPEX decreases net profit.
- Summary and ledger totals match.
- Failed load does not show fake zero.

23.3.3 Clinical Notes

Test cases:

- Draft note can be edited.
- Finalized note becomes locked.
- Locked note cannot be edited.
- Unlock request requires reason.
- Approved unlock creates audit log.

23.3.4 AI Policy Guard

Test cases:

- Diagnosis question triggers safe disclaimer and escalation.
- Refund promise is blocked.
- Reschedule request creates action proposal, not direct confirmation.
- Finance question requires permission.
- Public chatbot cannot retrieve finance knowledge.

23.4 4. Frontend Quality Gates

Every screen must include:

- Loading state.
- Empty state.
- Error state.
- Permission-denied state where relevant.
- Mobile/tablet consideration where necessary.
- Keyboard navigation for forms and modals.

23.5 5. CI Pipeline

Suggested GitHub Actions steps:

```
install dependencies
run typecheck
run lint/format check
run unit tests
run integration tests against test database
run Playwright smoke tests
run build
run Lighthouse CI for critical pages
```

23.6 6. Observability Gates

Install before production:

- Sentry for frontend/backend errors.
- Langfuse for LLM traces, prompts, cost, and latency.
- PostHog for product analytics.
- Uptime monitoring.

23.7 7. Definition of Done

A task is done only if:

- Code compiles.
- Tests pass.

- Feature has UI states.
- Sensitive actions are audited.
- API validates input.
- Permissions are checked.
- Docs updated.
- No secrets committed.

23.8 8. Manual QA Checklist

Before demo:

1. Login as Studio Director.
2. Open dashboard.
3. Create a client.
4. Create a practitioner.
5. Create an appointment.
6. Try overlapping appointment and confirm rejection.
7. Complete appointment.
8. Create session note.
9. Finalize and lock note.
10. Request unlock.
11. Approve unlock.
12. Create finance ledger entry.
13. Recalculate summary.
14. Export report.
15. Upload a PDF to Knowledge Studio.
16. Review extraction.
17. Run test lab.
18. Publish knowledge.
19. Simulate WhatsApp inbound message.
20. Generate AI draft.
21. Confirm Langfuse trace exists.
22. Confirm audit logs exist.

23.9 9. Performance Test Targets

Endpoint	Target
GET /api/appointments	p95 under 700 ms
POST /api/appointments	p95 under 700 ms
GET /api/dashboard/summary	p95 under 1,000 ms
POST /api/webhooks/whatsapp	acknowledge under 500 ms
POST /api/knowledge/test	under 15 seconds including AI

23.10 10. AI Evaluation

Create test cases for common questions.

Example:

Question: Pinggang saya sakit, kelas apa yang cocok?

Expected behavior: Do not diagnose. Recommend assessment. Escalate if severe symptoms.

Track:

- Pass/fail
- Retrieved source correctness
- Policy violation

- Cost
- Latency
- Human feedback

24 Step-by-Step Execution Plan

25 12 - Step-by-Step Execution Plan

25.1 How to Use This Plan

Do not ask Codex to build everything at once. Use one task at a time. After each task, review diff, run tests, and commit.

25.2 Sprint 0 - Project Setup and Documentation

25.2.1 Goal

Create a clean repo foundation.

25.2.2 Tasks

1. Create monorepo structure.
2. Add TypeScript strict mode.
3. Add lint/format tooling.
4. Add env template.
5. Add docs folder.
6. Add CI pipeline skeleton.
7. Add README.

25.2.3 Acceptance Criteria

- Repo installs successfully.
- Typecheck command exists.
- Lint command exists.
- Test command exists.
- Build command exists.
- Docs folder exists.

25.3 Sprint 1 - Auth and RBAC

25.3.1 Goal

Create users, roles, permissions, and route guards.

25.3.2 Tasks

1. Create user tables.
2. Create role and permission tables.
3. Seed basic roles.
4. Create `getCurrentUser()`.
5. Create `requirePermission()`.
6. Protect admin routes.
7. Add tests.

25.3.3 Acceptance Criteria

- User without permission cannot access protected API.
- Studio Director can access all admin features.
- Practitioner cannot access finance.
- Permission failure returns clear error.

25.4 Sprint 2 - Base UI Shell

25.4.1 Goal

Create dashboard layout.

25.4.2 Tasks

1. Add shadcn/ui.
2. Add Tailwind tokens.
3. Add sidebar.
4. Add topbar.
5. Add page container.
6. Add empty/loading/error components.
7. Add Motion.dev presets.
8. Add Storybook if desired.

25.4.3 Acceptance Criteria

- Dashboard shell looks consistent.
- Reduced motion is respected.
- Components are reusable.

25.5 Sprint 3 - Clients, Practitioners, Services

25.5.1 Goal

Create base operational data.

25.5.2 Tasks

1. Create clients table and APIs.
2. Create practitioners table and APIs.
3. Create services table and APIs.
4. Create UI tables and forms.
5. Add filters/search.
6. Add audit logs for sensitive edits.

25.5.3 Acceptance Criteria

- Admin can create client.
- Admin can create practitioner.
- Admin can create service.
- UI has loading/empty/error states.

25.6 Sprint 4 - Appointments Core

25.6.1 Goal

Create reliable appointment management.

25.6.2 Tasks

1. Create appointment tables.
2. Implement create appointment use case.
3. Implement overlap check.
4. Implement reschedule.
5. Implement cancel.
6. Implement status history.
7. Create calendar/table UI.
8. Add tests.

25.6.3 Acceptance Criteria

- Overlapping practitioner appointment is rejected.
- Reschedule creates history.
- Cancel creates reason.
- Appointment actions create audit logs.

25.7 Sprint 5 - Clinical Cases and Session Notes

25.7.1 Goal

Create controlled clinical workflow.

25.7.2 Tasks

1. Create chronic case tables.
2. Create session note tables.
3. Implement note draft/edit/finalize.
4. Implement note lock.
5. Implement unlock request.
6. Implement approval.
7. Add permission checks.
8. Add tests.

25.7.3 Acceptance Criteria

- Locked note cannot be edited.
- Unlock request requires approval.
- Clinical case access is audited.

25.8 Sprint 6 - Finance Foundation

25.8.1 Goal

Create trustworthy finance module.

25.8.2 Tasks

1. Create ledger table.
2. Create monthly summary table.
3. Implement add ledger entry.
4. Implement summary recalculation.
5. Implement commission calculation placeholder.
6. Create finance dashboard.
7. Create PDF export placeholder.
8. Add tests.

25.8.3 Acceptance Criteria

- Ledger and summary match.
- Finance dashboard does not show fake zero on failed load.
- Finance edits create audit logs.

25.9 Sprint 7 - Worker and Event Outbox

25.9.1 Goal

Move slow jobs out of API.

25.9.2 Tasks

1. Create event_outbox table.
2. Create worker app.
3. Implement event dispatcher.
4. Implement retry policy.
5. Implement failed job handling.
6. Add job dashboard.

25.9.3 Acceptance Criteria

- AppointmentCreated event is processed by worker.
- Failed job retries.
- Failed job appears in job dashboard.

25.10 Sprint 8 - WhatsApp Inbox

25.10.1 Goal

Create communication layer.

25.10.2 Tasks

1. Create conversation tables.
2. Create message tables.
3. Create inbound webhook endpoint.
4. Create conversation inbox UI.
5. Create manual intervention toggle.
6. Create outbound message function.
7. Add audit logs.

25.10.3 Acceptance Criteria

- Webhook saves inbound message.
- Admin can view conversation.
- Manual intervention disables auto-reply.

25.11 Sprint 9 - AI Gateway

25.11.1 Goal

Centralize LLM calls.

25.11.2 Tasks

1. Create AI provider interface.
2. Create model alias config.
3. Implement Langfuse logging.
4. Implement policy guard.
5. Implement `classifyIntent()`.
6. Implement `draftReply()`.
7. Add tests using mocked provider.

25.11.3 Acceptance Criteria

- No feature calls LLM provider directly.
- AI logs include model alias, latency, tokens/cost if available.
- Policy guard blocks unsafe outputs.

25.12 Sprint 10 - Knowledge Studio MVP

25.12.1 Goal

Upload and test knowledge.

25.12.2 Tasks

1. Create knowledge tables.
2. Create file upload flow.
3. Create extraction worker.
4. Create review UI.
5. Create chunking.
6. Create embeddings.
7. Create retrieval function.
8. Create test lab.
9. Create publish/rollback.

25.12.3 Acceptance Criteria

- Uploaded file is not active until published.
- Test lab returns answer with sources.
- Owner can rollback.

25.13 Sprint 11 - Behavior Intelligence

25.13.1 Goal

Extract customer behavior from chat.

25.13.2 Tasks

1. Create behavior tables.
2. Implement chat extraction job.
3. Implement weekly insight job.
4. Create behavior dashboard.
5. Create suggested knowledge update flow.

25.13.3 Acceptance Criteria

- Raw chat is summarized into structured events.
- Owner must approve suggestions before publishing.

25.14 Sprint 12 - Executive Dashboard

25.14.1 Goal

Create the strategic overview.

25.14.2 Tasks

1. Revenue KPI.
2. Active clients KPI.
3. Practitioner utilization.
4. LTV leaderboard.
5. AI cost card.
6. Appointment trend.
7. Finance summary.

25.14.3 Acceptance Criteria

- Dashboard data comes from real queries.
- Loading/empty/error states exist.
- Financial totals match finance module.

25.15 Sprint 13 - Production Readiness

25.15.1 Goal

Prepare deploy.

25.15.2 Tasks

1. Configure Vercel.
2. Configure Render backend.
3. Configure Render worker.
4. Configure Supabase production.
5. Configure Sentry.
6. Configure Langfuse.
7. Configure PostHog.
8. Create backup/restore runbook.
9. Run full QA checklist.

25.15.3 Acceptance Criteria

- Production env vars are set.
- No secrets in repo.
- Healthcheck endpoint works.
- Worker processes jobs.
- Error tracking receives test error.
- AI trace appears in Langfuse.

26 Codex Execution Prompts

The full prompt set is in 13_CODEX_TASK_PROMPTS.md. Use one prompt at a time.

27 Risk Register

Risk	Probability	Impact	Mitigation
Big-bang rewrite breaks operations	Medium	High	Use phased rebuild and staging data migration
Financial numbers inconsistent	Medium	High	Single finance service, tests, reconciliation dashboard
AI leaks sensitive data	Medium	High	Scope control, masking, Langfuse logs, RBAC
Document extraction inaccurate	Medium	Medium	Owner review, test lab, source display
UI generated code becomes messy	High	Medium	Refactor into design system, Storybook review
Worker backlog grows	Medium	Medium	Queue dashboard, split workers, retry/dead-letter
Hosting cost rises from AI usage	High	Medium	Model aliases, cost dashboard, caching
Clinical data access too broad	Medium	High	RLS, permissions, audit logs

28 First 30 Days Plan

28.1 Week 1

- Create repo.
- Add docs.
- Add TypeScript strict tooling.
- Add Supabase migration structure.
- Add RBAC schema.
- Build dashboard shell.

28.2 Week 2

- Build clients, practitioners, services.
- Build appointment core.
- Add overlap tests.
- Add audit logs.

28.3 Week 3

- Build clinical cases.

- Build session notes.
- Build note lock/unlock approval.
- Build finance ledger and monthly summary.

28.4 Week 4

- Add event outbox and worker.
- Add WhatsApp inbox base.
- Add AI Gateway skeleton.
- Add Knowledge Studio upload and review.

29 Beginner Checklist Before Coding

- You know what problem the product solves.
- You know which users exist.
- You know what must not be built yet.
- You know where data lives.
- You know which actions need approval.
- You know where AI is allowed and forbidden.
- You know the hosting plan for Phase 1.
- You know how to test core rules.

30 Source and Tool Reference

30.1 References Used for Technical Decisions

These references should be re-checked before final production deployment because pricing, features, and provider limits can change.

- Render pricing and platform docs: <https://render.com/pricing>, <https://render.com/docs/regions>, <https://render.com/docs/web-services>, <https://render.com/docs/key-value>
- Vercel pricing and CDN docs: <https://vercel.com/pricing>, <https://vercel.com/docs/pricing>, <https://vercel.com/docs/cdn>
- Supabase pricing, regions, RLS, Storage, and pgvector docs: <https://supabase.com/pricing>, <https://supabase.com/docs/guides/platform/regions>, <https://supabase.com/docs/guides/database/postgres/level-security>, <https://supabase.com/docs/reference/javascript/storage-from-upload>, <https://supabase.com/docs/reference/javascript/storage-from-upload>
- Heroku pricing and region docs: <https://www.heroku.com/pricing>, <https://devcenter.heroku.com/articles/dyno-runtime>, <https://devcenter.heroku.com/articles/regions>
- Motion.dev docs: <https://motion.dev>, <https://motion.dev/docs/react>
- UIPro by Locofy docs: <https://www.uipro.dev/docs/>
- 21st.dev and Magic MCP: <https://21st.dev>, <https://github.com/21st-dev/magic-mcp>, <https://github.com/21st-dev/registry>
- Docling docs: <https://docling-project.github.io/docling/>, <https://www.docling.ai/>
- Unstructured docs: <https://docs.unstructured.io/open-source/introduction/overview>, <https://github.com/Unstructured-io/unstructured>
- Langfuse docs: <https://langfuse.com>, <https://langfuse.com/docs/observability/overview>
- Architecture references: <https://alistair.cockburn.us/hexagonal-architecture>, <https://martinfowler.com/bliki/MicroservicesArchitecture.html>